Introducción a la manipulación de documentos Web mediante lenguajes de script de cliente. JavaScript.

1 Introducción 2
2 Lenguajes de script de cliente. Características y sintaxis básica de JavaScript 2
2.1 Inclusión de código de cliente en contenido web 3
2.1.1 Incrustación de código en línea (Inline JavaScript) 3
2.1.2 Inclusión de scripts externos 3
2.1.3 Scripts en atributos HTML (Event Handlers en línea) 4
2.2 Variables y constantes 4
2.3 Tipos de datos 5
2.4 Operadores 5
2.5 Estructuras de control 6
2.6 Bucles (Loops) 7
2.7 Funciones 8
3 Selección y acceso a elementos. 8
3.1 Métodos más comunes para seleccionar elementos 9
3.1.1 getElementById() 9
3.1.2 getElementsByClassName() 9
3.1.3 getElementsByTagName() 9
3.1.4 querySelector() 9
3.1.5 querySelectorAll() 10
3.2 Acceso a propiedades y atributos de los elementos 10
3.2.1 Modificar el contenido textual 10
3.2.2 Acceso y modificación de atributos 11
3.2.3 Modificar estilos en línea 11
4 Creación de elementos. 12
4.1 document.createElement() 12
4.2 appendChild() 12
4.3 insertBefore() 12
5 Eliminación de elementos. 13
5.1 removeChild() 13
5.2 remove() 14
5.3 Vaciar el contenido de un elemento usando innerHTML 14
6 Gestión de eventos 14
6.1 addEventListener() 15
6.2 Atributos HTML on-event 16
6.3 Propiedades de eventos del DOM 17

1 Introducción

Por manipulación de documentos web en la parte cliente (normalmente el *navegador)*, entendemos la capacidad de modificar y controlar el contenido de una página web mediante lenguajes de programación que se ejecutan en el parte del cliente. Estos lenguajes permiten alterar tanto la estructura como el comportamiento de una página sin necesidad de recargarla, lo que mejora la interactividad, el rendimiento y la experiencia del usuario.

Los lenguajes de script del lado del cliente se ejecutan directamente en el navegador del usuario y no requieren interacción con el servidor para realizar cambios en la página web. El más utilizado es **JavaScript,** que se puede considerar como un estándar a día de hoy, ya que ha conseguido que el uso de otros lenguajes como *TypeScript*, *Dart* o *Elm*, sea mínimo.

Un concepto muy importante cuando trabajamos con *JavaScript* es el de *DOM (Modelo de Objetos del Documento)*. El ***DOM (Modelo de Objetos del Documento)*** es una representación estructurada de una página web que permite acceder y modificar el contenido y estructura de un documento HTML o XML mediante lenguajes de programación. A través del *DOM*, los lenguajes de script pueden:

- Acceder a elementos HTML específicos.
- Modificar el contenido o el estilo de los elementos.
- Añadir o eliminar elementos del documento.
- Reaccionar a eventos del usuario (clics, teclas, movimientos del ratón, etc.).

El *DOM* organiza el documento en una estructura jerárquica similar a un árbol donde cada nodo representa un elemento de la página.

2 Lenguajes de script de cliente. Características y sintaxis básica de JavaScript

Como hemos comentado, los ***lenguajes de script*** de cliente (*JavaScript* esencialmente) son aquellos que se ejecutan en el navegador web del usuario, en lugar de en el servidor. Permiten gestionar la interactividad del usuario y manipular el contenido del documento sin recargar la página. Sus características básicas son:

- **Ejecución en el navegador**: Los lenguajes de script del lado del cliente se ejecutan directamente en el navegador del usuario, lo que permite una respuesta inmediata a las interacciones del usuario.

- **Manipulación del DOM**: Permiten acceder y modificar el **Document Object Model (DOM)**, que es la representación estructurada de una página web.

- **Interactividad**: Posibilitan reaccionar a eventos del usuario, como clics, desplazamientos y pulsaciones de teclas.

- **Estándares**: El comportamiento de los lenguajes de script del lado del cliente está regulado por estándares como ***ECMAScript*** para *JavaScript*, que es mantenido por ***ECMA International***.

JavaScript es un lenguaje de programación dinámico, ***interpretado***, y de ***tipado débil***, **lo que significa que no es necesario declarar los tipos de variables antes de utilizarlas, y las variables pueden cambiar de tipo en tiempo de ejecución**.

2.1 Inclusión de código de cliente en contenido web

JavaScript ofrece múltiples mecanismos para integrarse en el contenido web:

- **Incrustación de código en línea** (directamente en el archivo HTML).
- **Inclusión de scripts externos**.
- **Uso de atributos en elementos HTML**.

2.1.1 Incrustación de código en línea (Inline JavaScript)

La forma más directa de incluir código JavaScript en un documento HTML es escribiendo el código dentro de una etiqueta ***<script>*** en la misma página web. Este método se denomina **JavaScript en línea**.

```
<script>
  // Aquí va el código JavaScript en línea
  alert("Hola, esto es un script en línea");
</script>
```

Este código se puede colocar dentro del archivo HTML en dos ubicaciones principales:

- **En la sección *<head>***: El script se ejecuta mientras la página se está cargando. OJO, puede afectar el rendimiento, ya que el código debe ejecutarse antes de que la página termine de cargarse.
- **En la sección *<body>***: El script se ejecuta en el punto donde aparece dentro del documento.

2.1.2 Inclusión de scripts externos

Otra forma común de incluir JavaScript es mediante la referencia a archivos de script externos. Estos archivos se guardan en un archivo *.js* separado y se enlazan desde el archivo HTML. Esto permite organizar mejor el código y reutilizarlo en múltiples páginas.

```
<script src="ruta/del/archivo.js"></script>
```

En proyectos de cierto tamaño, este mecanismo permite separar el HTML del código JavaScript, lo que puede incidir en una mejora de la organización y el mantenimiento del código.

Muchas veces se utiliza JavaScript a través de bibliotecas o frameworks que facilitan la escritura de código interactivo y dinámico. Algunas de las más populares son ***jQuery***, ***React***, ***Vue.js*** y ***BootStrap***. Las bibliotecas de JavaScript permiten extender las capacidades del lenguaje y facilitar tareas comunes, como manipulación del DOM, gestión de eventos, o integración de datos.

```
<head>
  <meta charset="UTF-8">
  <title>Incluir jQuery</title>
  <!-- Incluir jQuery desde un CDN -->
  <script src="https://code.jquery.com/jquery-3.6.0.min.js"></script>
</head>
```

2.1.3 Scripts en atributos HTML (Event Handlers en línea)

También, se puede incluir código JavaScript directamente en los **atributos HTML** que manejan eventos. Estos se conocen como **manejadores de eventos en línea**, y están presentes en elementos HTML como *onclick*, *onmouseover*, *onchange*, entre otros.

Los veremos más adelante, pero su uso no suele estar recomendado.

2.2 Variables y constantes

Las variables en JavaScript permiten almacenar datos que luego podemos manipular. Como hemos comentado en un punto anterior, JavaScript no requiere indicar su tipo de datos, ya que éste se determina dinámicamente en el momento de la asignación del dato.

Hay tres formas principales de declarar variables: ***var***, ***let*** y ***const***.

- ***var***: Era la forma original de declarar variables, pero tiene un comportamiento impredecible debido a su **alcance de función** (hoisting). Su uso se desaconseja en favor de ***let*** y ***const***.
- ***let***: Es la manera moderna de declarar variables. Tiene un **alcance de bloque** (sólo está disponible dentro del bloque { } en el que se define).
- ***const***: Similar a *let*, pero se usa para declarar **constantes**; es decir, valores que no pueden ser reasignados después de su inicialización. Sin embargo, los *objetos* y *arrays* declarados con *const* pueden ser modificados en su contenido.

```
let nombre = "Ana";      // Variable que puede cambiar
const PI = 3.1416;       // Constante, su valor no puede cambiar
```

2.3 Tipos de datos

JavaScript tiene varios tipos de datos primitivos:

- ***Number***: Números, tanto enteros como decimales.

  ```
  let numero = 42;
  let decimal = 3.14;
  ```

- ***String***: Cadenas de texto. Se pueden usar comillas simples (' ') o dobles (" ").

  ```
  let saludo = "Hola, mundo!";
  let nombre = 'Juan';
  ```

  También, es posible usar las llamadas **comillas invertidas** o **backticks** (`), que van a permitirnos interpolar variables dentro de una cadena, es decir, insertar sus valores dentro del texto. Para interpolar una variable usaremos la sintaxis **${x}**, donde x es la variable de la que queremos sacar el valor. Por ejemplo:

  ```
  alert(`La hora actual es: ${horas}:${minutos}:${segundos}`);
  ```

- ***Boolean***: Valores lógicos, true o false.

  ```
  let esMayorDeEdad = true;
  let esEstudiante = false;
  ```

- ***null***: Un valor intencionalmente vacío.

  ```
  let valorNulo = null;
  ```

- ***Undefined***: El valor por defecto de una variable no inicializada.

  ```
  let sinDefinir;
  ```

- ***Object***: Colección de propiedades y valores.

  ```
  let persona = { nombre: "Carlos",  edad: 30  };
  ```

- ***Array***: Estructura que contiene múltiples valores en un orden específico.

  ```
  let frutas = ["manzana", "naranja", "plátano"];
  ```

2.4 Operadores

JavaScript ofrece una variedad de operadores para realizar operaciones matemáticas, lógicas y de comparación.

- **Operadores aritméticos**:

```
let suma = 10 + 5;       // 15
let resta = 10 - 5;      // 5
let multiplicacion = 10 * 5;  // 50
let division = 10 / 5;    // 2
let modulo = 10 % 3;      // 1
let incremento = ++suma;  // Incrementa en 1
let decremento = --resta; // Decrementa en 1
```

- **Operadores de asignación:**

```
let x = 10;    // Asignación
x += 5;        // Suma y asigna: x = x + 5
x -= 3;        // Resta y asigna: x = x - 3
x *= 2;        // Multiplica y asigna: x = x * 2
x /= 4;        // Divide y asigna: x = x / 4
```

- **Operadores de comparación**:
  - ***Igualdad estricta (===)***: Compara tanto valor como tipo.
  - ***Igualdad débil (==):*** Compara sólo el valor
  - ***Diferencia estricta (!==)***: Compara valor y tipo.
  - ***Diferencia débil (!=):*** Compara sólo el valor.
  - ***Mayor (>), menor (<), mayor o igual (>=), menor o igual(<=)***

- **Operadores lógicos**:

```
let resultado = (a > 3 && b < 10);  // AND lógico
resultado = (a > 3 || b < 10);      // OR lógico
resultado = !(a > 3);               // NOT lógico
```

2.5 Estructuras de control

Las **estructuras de control** permiten modificar el flujo de ejecución del código en función de condiciones.

- **Condicionales (if, else if, else)**:

```
let edad = 18;
if (edad >= 18) {
   console.log("Es mayor de edad");
} else if (edad >= 13) {
   console.log("Es adolescente");
} else {
   console.log("Es menor de edad");
}
```

- **Operador ternario**: Es una forma concisa de escribir una condición simple (*if else*)I

```
let mensaje = (edad >= 18) ? "Adulto" : "Menor de edad";
```

Sería equivalente a:

```
if (edad>=18) {
   mensaje=”Adulto”
}
else {
   mensaje=”Menor de edad”
}
```

- **Estructura switch**: Se utiliza para evaluar una variable contra múltiples valores posibles. La cláusula default sólo se tendrá en cuenta si no se cumple ninguna de las anteriores

```
let dia = 3;
switch(dia) {
   case 1:
      console.log("Lunes");
      break;
   case 2:
      console.log("Martes");
      break;
   case 3:
      console.log("Miércoles");
      break;
   default:
      console.log("Día no válido");
}
```

2.6 Bucles (Loops)

Los bucles permiten ejecutar un bloque de código repetidamente hasta que se cumpla una condición.

- **while**: Ejecuta un bloque de código mientras una condición sea verdadera.

```
let i = 0;
while (i < 5) {
   console.log("Iteración " + i);
   i++;
}
```

- **for**: Se usa para repetir un bloque de código un número específico de veces.

```
for (let i = 0; i < 5; i++) {
    console.log("Iteración número " + i);
}
```

- **do...while**: Similar al *while*, pero ejecuta al menos una vez el código antes de verificar la condición.

```
let j = 0;
do {
    console.log("Iteración " + j);
    j++;
} while (j < 5);
```

2.7 Funciones

Las **funciones** en JavaScript son bloques de código reutilizables que se pueden invocar en cualquier parte del programa. Los **parámetros** que recibe opcionalmente una función tienen que ponerse entre paréntesis, de manera que cuando invoquemos a la función, le pasemos los datos con los que queremos trabajar.

```
function sumar(a, b) {
    return a + b;
}
let resultado = sumar(5, 3);  // 8
```

Existe una forma más concisa de escribir funciones en JavaScript que es lo que se conoce como **funciones anónimas de una sola línea** o **funciones flecha,** muy parecidas a las **funciones lambda** de otros lenguajes de programación.

```
let resta = (a, b) => a - b;
// a y b son los parámetros y lo que viene después de la flecha es el código
console.log(resta(10, 5));
// La función anónima puede ser invocada a través de la variable asociada
```

3 Selección y acceso a elementos.

El ***DOM*** es una representación estructurada de la página web, donde cada elemento HTML es un nodo dentro de una estructura jerárquica. Para manipular el *DOM* mediante JavaScript, primero necesitamos **seleccionar** los elementos a los que queremos acceder o modificar. JavaScript proporciona varias maneras de seleccionar elementos del *DOM* usando métodos del objeto ***document***.

3.1 Métodos más comunes para seleccionar elementos

3.1.1 getElementById()

Este método selecciona un **único elemento** basado en el valor del atributo ***id*** que se le haya asignado en el HTML. Dado que los *id* son únicos en un documento HTML, siempre se seleccionará un solo elemento.

```
<div id="cabecera">Bienvenido a mi página</div>

<script>
    const cabecera = document.getElementById("cabecera");
    console.log(cabecera.textContent);  // Imprime: Bienvenido a mi página
</script>
```

3.1.2 getElementsByClassName()

Este método selecciona **todos los elementos** que tienen una clase específica. Devuelve una ***HTMLCollection***, una lista similar a un array, pero no exactamente igual.

```
<p class="texto">Primero</p>
<p class="texto">Segundo</p>
<p class="texto">Tercero</p>

<script>
 const textos = document.getElementsByClassName("texto");
 console.log(textos[0].textContent);  // Imprime: Primero
</script>
```

3.1.3 getElementsByTagName()

Este método selecciona **todos los elementos** con una etiqueta HTML específica (como div, p, h1, etc.). También devuelve una **HTMLCollection**.

```
<p>Primero</p>
<p>Segundo</p>
<script>
 const parrafos = document.getElementsByTagName("p");
 console.log(parrafos.length);  // Imprime: 2 (número de elementos <p>)
</script>
```

3.1.4 querySelector()

El método ***querySelector()*** devuelve el **primer elemento** que coincida con un **selector CSS**. Este método es muy flexible ya que acepta selectores complejos como *#id*, *.clase*, *elemento.clase*, etc.

```
<div class="contenedor">Contenido principal</div>
<script>
 const contenedor = document.querySelector(".contenedor");
 console.log(contenedor.textContent);  // Imprime: Contenido principal
</script>
```

3.1.5 querySelectorAll()

Este método selecciona **todos los elementos** que coincidan con un **selector CSS** y devuelve un ***NodeList***, una lista de nodos que se puede recorrer fácilmente con el método ***forEach***.

```
<div class="caja">Caja 1</div>
<div class="caja">Caja 2</div>
<div class="caja">Caja 3</div>
<script>
    const cajas = document.querySelectorAll(".caja");
    cajas.forEach(caja => {
      console.log(caja.textContent);  // Imprime los textos: Caja 1, Caja 2, Caja 3
   });
</script>
```

3.2 Acceso a propiedades y atributos de los elementos

Una vez seleccionado un elemento, puedes acceder o modificar sus propiedades, como su **texto**, **atributos** o **estilos**.

3.2.1 Modificar el contenido textual

Existen una serie de propiedades que permiten cambiar el contenido de un elemento:

- ***textContent***: Devuelve o establece el texto dentro de un elemento.

  ```
  const titulo = document.querySelector("h1");
  titulo.textContent = "Nuevo título";  // Cambia el texto a "Nuevo título"
  ```

- ***innerHTML***: Devuelve o establece el contenido HTML dentro de un elemento. Con *innerHTML* podemos insertar elementos HTML dentro de un contenedor.

  ```
  const div = document.querySelector("#contenedor");
  div.innerHTML = "<p>Texto nuevo con HTML</p>"; // Inserta un párrafo
  ```

3.2.2 Acceso y modificación de atributos

- ***getAttribute*() y *setAttribute***(): Se usan para obtener y modificar los atributos de un elemento HTML, como ***src*** (en imágenes), ***href*** (en enlaces), ***alt***, etc.

```
const enlace = document.querySelector("a");
enlace.setAttribute("href", "https://www.nuevaurl.com");  // Cambia el href
```

- ***classList***: Permite acceder y modificar las *clases* de un elemento HTML de manera más eficiente. Con *classList*, podemos añadir, quitar o alternar *clases* sin sobrescribir el atributo *class*.

```
const boton = document.querySelector(".boton");
// Añade la clase "activo"
boton.classList.add("activo");
// Elimina la clase "inactivo"
boton.classList.remove("inactivo");
// Alterna la clase "activo" (la añade si no está, la quita si está)
boton.classList.toggle("activo");
```

3.2.3 Modificar estilos en línea

El atributo *style* permite acceder y cambiar los estilos en línea de un elemento. A través de esta propiedad, **puedes ajustar cualquier estilo CSS**. Por ejemplo, puedes cambiar el color, tamaño de fuente, etc. Su sintaxis es:

```
elemento.style.propiedadCSS = "valor";
```

Por ejemplo:

```
const parrafo = document.querySelector("p");
parrafo.style.color = "red";  // Cambia el color del texto a rojo
parrafo.style.fontSize = "20px";  // Cambia el tamaño de la fuente a 20px
```

En JavaScript, las propiedades CSS cuyo nombre incluye guiones (-) se convierten a atributos de *style* con una nomenclatura de **camelCase**. Ejemplos:

- *background-color* → *backgroundColor*
- *font-size* → *fontSize*
- *margin-left* → *marginLeft*

Para cambiar varios estilos en una única sentencia podemos usar el atributo *cssText*, que nos permitirá pasarle un texto con el código CSS que queramos ejecutar (OJO, en este caso se usaría la notación habitual de CSS)

```
const parrafo = document.getElementById('miParrafo');
parrafo.style.cssText = "color: red; font-size: 18px; background-color: yellow;";
```

4 Creación de elementos.

La manipulación del contenido del *DOM* es uno de los pilares del desarrollo web dinámico. JavaScript nos permite crear nuevos elementos HTML, modificar sus propiedades y contenido, y luego insertarlos en el *DOM*. A continuación, exploraremos cómo realizar estas acciones en detalle.

4.1 document.createElement()

Este método crea un nuevo elemento HTML, pero no lo añade automáticamente al DOM. Posteriormente, usamos otros métodos para insertarlo en la estructura existente. Por ejemplo:

```
const parrafo = document.createElement('p');
parrafo.textContent = "Este es un nuevo párrafo";
```

En este ejemplo, hemos creado un nuevo párrafo en memoria, pero aún no lo hemos añadido al documento HTML.

4.2 appendChild()

Este método añade el nodo que hemos creado como **último hijo de un elemento padre** existente. Por ejemplo:

```
const contenedor = document.getElementById('contenedor');
contenedor.appendChild(parrafo); // Añade el nuevo párrafo al contenedor
```

4.3 insertBefore()

Si necesitas insertar el nuevo elemento **antes de un nodo específico** (es decir, como hermano anterior), puedes usar el método ***insertBefore*()**.

```
const contenedor = document.getElementById('contenedor');   // Padre
const referencia = document.getElementById('referencia');      // Hermano
contenedor.insertBefore(parrafo, referencia);  // Añade el párrafo antes
```

En JavaScript **no existe** un método nativo llamado **insertAfter()**. Para insertar un elemento después de otro en JavaScript, sin embargo, podemos utilizar el método ***insertBefore*()** en combinación con la propiedad ***nextSibling,*** que referencia el siguiente nodo hermano de un elemento.

```
// Si el elementoReferencia tiene un hermano a la derecha, insertamos antes
if (elementoReferencia.nextSibling) {
    padre.insertBefore(nuevoElemento, elementoReferencia.nextSibling);
} else {
    // Si no hay un siguiente hermano, simplemente añadimos al final del padre
    padre.appendChild(nuevoElemento);
}
```

5 Eliminación de elementos.

Eliminar elementos del **DOM** implica quitar nodos HTML existentes de la estructura del documento. JavaScript ofrece métodos específicos para hacer esto de manera eficiente, permitiendo tanto eliminar elementos concretos como borrar grupos de nodos.

5.1 removeChild()

El método ***removeChild*()** se usa para eliminar un nodo hijo específico de su nodo padre. En este caso, debes identificar tanto el nodo hijo que deseas eliminar como el nodo padre que lo contiene. Su sintaxis es:

```
padre.removeChild(hijo);
```

Por ejemplo:

```
<!DOCTYPE html>
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Eliminar con removeChild()</title>
</head>
<body>
  <div id="contenedor">
    <p id="parrafo1">Primer párrafo</p>
    <p id="parrafo2">Segundo párrafo</p>
    <p id="parrafo3">Tercer párrafo</p>
  </div>
```

```
    <script>
      // Seleccionar el elemento padre y el nodo hijo que queremos eliminar
      const contenedor = document.getElementById('contenedor');
      const parrafo2 = document.getElementById('parrafo2');

      // Eliminar el segundo párrafo
      contenedor.removeChild(parrafo2);
    </script>
</body>
</html>
```

5.2 remove()

El método ***remove()*** es más sencillo y directo que *removeChild(),* ya que **no requiere que especifiques el nodo padre**. Simplemente puedes llamar a este método en el propio nodo que deseas eliminar, lo que lo hace más cómodo en muchas situaciones. Por ejemplo:

```
<script>
    // Seleccionar el párrafo
    const parrafo1 = document.getElementById('parrafo1');

    // Eliminar el párrafo
    parrafo1.remove();
</script>
```

5.3 Vaciar el contenido de un elemento usando innerHTML

En lugar de eliminar un solo elemento, es posible que quieras **eliminar todo el contenido** de un contenedor. Para esto, puedes asignar una cadena vacía al atributo ***innerHTML*** del contenedor, lo que eliminará todo su contenido HTML.

```
elemento.innerHTML = "";
```

6 Gestión de eventos

La **gestión de eventos en JavaScript** es una de las características más importantes cuando se trata de crear aplicaciones web interactivas y dinámicas. Los eventos permiten que una página web reaccione a acciones de los usuarios, como hacer clic en un botón y **son el corazón de la interacción en el DOM**.

Un **evento** es cualquier interacción que ocurra en el navegador o en un elemento HTML de una página. Esto puede ser desde hacer clic en un botón, mover el ratón sobre un elemento, presionar una tecla, cambiar el valor de un formulario, entre muchos otros. Los eventos permiten capturar estas interacciones y responder con acciones programadas. Algunos ejemplos comunes de eventos son:

- **Eventos de ratón**: *click, dblclick, mouseover, mouseout, mousemove*.
- **Eventos de teclado**: *keydown, keyup, keypress*.
- **Eventos de formulario**: *submit, change, input, focus, blur*.
- **Eventos de ventana**: *load, resize, scroll*.

Existen varias formas de gestionar eventos en JavaScript:

- ***addEventListener()***.
- Atributo HTML **on-event**.
- Propiedades de evento del DOM (como *onclick*, *onmouseover*).

6.1 addEventListener()

El método ***addEventListener*()** es el enfoque más flexible y recomendado para la gestión de eventos. Este método permite asociar un evento a un elemento sin sobrescribir otros eventos que el mismo elemento pueda tener. Su sintaxis es:

```
elemento.addEventListener('tipoDeEvento', funcionManejadora);
```

Donde:

- **elemento**: El nodo del DOM al que se le está asignando el evento.
- **tipoDeEvento**: Es el tipo de evento que deseas capturar (por ejemplo, *click*, *mouseover*, etc.).
- **funcionManejadora**: Es la función que se ejecutará cuando ocurra el evento.

Por ejemplo:

```
<script>
  const boton = document.getElementById('miBoton');

  // Añadir el evento de clic
  boton.addEventListener('click', function() {
    alert("¡Botón pulsado!");
  });
</script>
```

Las ventajas que presenta el uso de *addEventListener()* son:

- **Permite añadir múltiples eventos del mismo tipo a un mismo elemento** sin que se sobrescriban.
- Es la forma más moderna y ampliamente soportada en todos los navegadores.
- Puedes eliminar eventos con ***removeEventListener*()** si es necesario.

Las **funciones que responden a eventos en JavaScript** pueden recibir parámetros. De hecho, siempre reciben un **objeto de evento *(Event)*** como parámetro, aunque no lo declaremos explícitamente. Este objeto contiene información detallada sobre el evento que se acaba de producir, como su tipo, el elemento que lo generó, la posición del puntero del ratón, las teclas que se presionaron, entre muchas otras propiedades. Si, por ejemplo, quisiéramos acceder a la posición del puntero del ratón cuando ocurre un evento, podríamos usar las propiedades del objeto *MouseEvent*. Este objeto es una subclase del objeto general *Event*, y está disponible cuando se manejan eventos de ratón (como *click*, *mousemove*, *mouseover*, etc.). Por ejemplo:

```
<script>
  const caja = document.getElementById('caja');
  const coordenadas = document.getElementById('coordenadas');

  // Escuchar el evento 'mousemove' para obtener la posición del ratón
  caja.addEventListener('mousemove', function(evento) {
    // Obtener las coordenadas relativas a la ventana
    const x = evento.clientX;
    const y = evento.clientY;

    // Mostrar las coordenadas en la página
    coordenadas.textContent = `Posición: (X: ${x}, Y: ${y})`;
  });
</script>
```

El objeto ***Event*** tiene también métodos importantes que pueden resultarnos muy útiles en ciertas situaciones:

- ***preventDefault***(): Cancela el comportamiento predeterminado del evento (por ejemplo, evitar que un enlace navegue a una URL o que un formulario se envíe).
- ***stopPropagation***(): Evita que el evento se propague a otros elementos.

6.2 Atributos HTML on-event

Una forma más antigua de manejar eventos es utilizando atributos de eventos directamente en los elementos HTML. Estos atributos tienen nombres que comienzan con *on*, como *onclick*, *onmouseover*, *onchange*, etc.

```
<html lang="es">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>Evento click con atributo HTML</title>
</head>
<body>
  <!-- Asignar la función manejadora directamente en el atributo -->
  <button onclick="alert('¡Botón clickeado!')">Haz clic aquí</button>
</body>
</html>
```

Este método presenta dos claras desventajas, por lo que deberíamos tratar de evitar su uso:

- **Limita la separación de la lógica del HTML y el JavaScript**, lo que hace que el código sea menos modular y más difícil de mantener.
- **Sobrescribe otros manejadores de eventos si los defines para el mismo evento**.

6.3 Propiedades de eventos del DOM

Otra manera de gestionar eventos es asignar una función manejadora directamente a una propiedad de evento del DOM, como *onclick*, *onmouseover*, etc. Por ejemplo:

```
<html lang="es">
<head>
  <title>Evento con propiedad DOM</title>
</head>
<body>
  <button id="miBoton">Haz clic aquí</button>

  <script>
    const boton = document.getElementById('miBoton');

    // Asignar la función manejadora directamente a la propiedad onclick
    boton.onclick = function() {
      alert("¡Botón pulsado!");
    };
  </script>
</body>
</html>
```

Esté método de gestionar eventos presenta dos desventajas respecto al de *addEventListener*() que son:

- Como ocurre con los atributos HTML *on-event*, si asignas otro evento al mismo *onclick*, sobrescribirás el anterior, lo que limita su flexibilidad.
- **No** permite añadir múltiples manejadores para el mismo evento.